C# MİMARİ VE BELLEK BAŞVURU KILAVUZU

C# Diziler, Metotlar ve Bellek Yönetimi (`ref` & İleri Düzey)

Stack vs. Heap, 1D/2D/Jagged Diziler, Parametre Aktarım Modelleri ve Kod Analizi

1. Dizi (Array) Nedir ve Bellekte Nasıl Yaşar?

Dizi, **aynı türdeki** verileri bellekte peş peşe (contiguous - bitişik blok halinde) saklayan sabit boyutlu bir veri yapısıdır. C#'ta diziler **Referans Tipidir (Reference Type)**:

Stack (Yığın) Tarafı: Referans Kartviziti

Dizi tanımlandığında (`int[] sayilar;`), Stack üzerinde yalnızca Heap'teki adresi (işaretçiyi) tutacak 4 veya 8 baytlık bir alan ayrılır. Değer atanmadığında varsayılanı `null`'dır.

Heap (Öbek) Tarafı: Gerçek Hücreler

`new int[4]` dendiğinde Heap'te 4 adet tam sayı sığacak kesintisiz bir bellek bloğu açılır ve başlangıç adresi (örn: 0x2A1B) Stack'teki değişkene teslim edilir.

2. Dizi Türleri ve Tanımlama Biçimleri

C# dilinde kullanım amacına göre üç temel dizi mimarisi mevcuttur. Her birinin bellek yerleşimi ve döngü sınırları farklı metotlarla denetlenir:

```
// 1. Tek Boyutlu Dizi (1D Array) int[] sayilar = new int[3] { 10, 20, 30 }; // Boyut: sayilar.Length // 2. İki Boyutlu Matris Dizi (2D Rectangular Array - Tüm satırlar eşit sütunludur) string[,] sinemaSalonu = new string[3, 4]; // 3 Satır, 4 Sütun sinemaSalonu[0, 0] = "[X]"; // Döngü sınırları: sinemaSalonu.GetLength(0) -> Satır, sinemaSalonu.GetLength(1) -> Sütun // 3. Düzensiz Dizi (Jagged Array - Dizilerin Dizisi, her satır farklı uzunluktadır) int[][] siniflar = new int[2][]; siniflar[0] = new int[3] { 1, 2, 3 }; // 0. satır 3 elemanlı siniflar[1] = new int[5] { 4, 5, 6, 7, 8 }; // 1. satır 5 elemanlı
```

3. Dizi Mimarilerinin Karşılaştırması

1D Dizi

Yatay doğrusal liste. `arr.Length` ile toplam eleman alınır. Hızlıdır ve doğrudan indekslenir.

2D Matris Dizi

Excel tablosu gibi nizamidir. `GetLength(0)` satırı, `GetLength(1)` sütunu verir.

Jagged Dizi

RAM israfını önler. Her satır bağımsız bir dizidir; `arr[i].Length` kullanılır.

İndeks Hatası: N elemanlı bir dizinin son eleman indeksi daima `N - 1`'dir. `arr[N]` çağrısı derleme anında değil, çalışma anında doğrudan `IndexOutOfRangeException` fırlatır.

4. Metotların Anatomisi ve Türleri

Metotlar; belirli bir algoritmayı paketleyen, tek bir görevi üstlenen (Single Responsibility) ve gerektiğinde parametre alıp geriye sonuç dönen bağımsız kod bloklarıdır:

Metot Türü	Geri Dönüş	Parametre Durumu	Örnek Sözdizimi
İş Yapan (Prosedür)	`void`	Parametresiz	`void EkraniTemizle() { Console.Clear(); }`
Veri Alan İşçi	`void`	Parametreli	`void LogYaz(string mesaj) { ... }`
Veri Üreten Kaynak	Tür Belirtilir	Parametresiz	`int ZarAt() { return rnd.Next(1, 7); }`
Dönüştürücü / Fonksiyon	Tür Belirtilir	Parametreli	`decimal KdvHesapla(decimal tutar) { return tutar * 0.20m; }`

5. Metot İmzası (Method Signature) ve Overloading

Bir sınıfta aynı isimde birden fazla metot bulunmasına **Method Overloading (Aşırı Yükleme)** denir. Derleyici, hangi metodun çalışacağına yalnızca *Metot İmzası* ile karar verir:

```
// İMZA KURALI: Metot Adı + Parametre Tipleri + Parametre Sayısı + Sıralama public void Yazdir(int sayi) { Console.WriteLine("Tam Sayı: " + sayi); } public void Yazdir(string metin) { Console.WriteLine("Metin: " + metin); } public void Yazdir(string metin, int tekrar) { /* 2 parametreli versiyon */ } // DİKKAT: Sadece geri dönüş tipinin farklı olması (int vs void) overload SAĞLAMAZ! // int Yazdir(int sayi) { ... } -> DERLEME HATASI! Parametre listesi yukarıdakiyle aynıdır.
```

6. İleri Düzey Parametreler: `params` ve İsteğe Bağlı Parametreler

Kaç parametre geleceğinin bilinmediği durumlarda `params`, varsayılan değer atanmak istendiğinde opsiyonel parametreler kullanılır:

```
// 1. params: Sayısı belirsiz argümanları tek bir diziye toplar (Listenin sonunda olmalıdır) decimal Topla(params decimal[] sayilar) { decimal toplam = 0; foreach (var s in sayilar) toplam += s; return toplam; } // Çağrılış: Topla(10, 20); veya Topla(5, 15, 25, 35, 45); // 2. Optional Parameters: Varsayılan değer tanımlama (En sonda yer almalıdır) void BaglantiKur(string host, int port = 8080, bool ssl = true) { /* ... */ }
```

> **Kurallar:** Bir metotta en fazla bir adet `params` bulunabilir. Hem `params` hem de varsayılan değerli parametreler metot imzasında daima en sağda (son sırada) olmak zorundadır.

7. Değer vs. Referans Aktarımı (`Pass by Value` vs. `Pass by Reference`)

C# dilinde değişkenlerin metotlara aktarılış biçimi, belleğin Stack ve Heap davranışlarını doğrudan etkiler:

DEĞER İLE AKTARIM (VARSAYILAN)

Kopya Mekanizması

Değişkenin Stack'teki değeri okunur ve metodun yerel parametre değişkenine *fotokopi* olarak kopyalanır. Metot bu kopya üzerinde ne yaparsa yapsın, çağıran yerdeki asıl değişken sabit kalır.

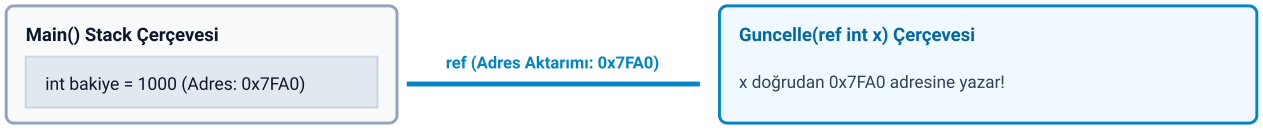
REFERANS İLE AKTARIM (`ref`)

Adres Mekanizması

Kopya üretilmez. Stack'teki asıl bellek hücresinin doğrudan fiziksel adresi metoda iletilir. Metot değişkenin içeriğini değiştirdiğinde, çağıran taraftaki orijinal kutu doğrudan güncellenir.

8. Bellek Şeması: `ref` Sayesinde Stack Adresinin Paylaşılması

Aşağıdaki şemada, `Degistir(ref sayi)` çağrıldığında bellekte yeni bir bağımsız alan açılmadığı, aynı adresin paylaşıldığı gösterilmiştir:



9. Parametre Değiştiricileri: `ref`, `out`, `in` Karşılaştırması

Belirteç	Girişte Değer Zorunlu mu?	Gövdede Atama Şart mı?	Değeri Değiştirebilir mi?	Mühendislik Amacı
Normal	Evet	Hayır	Hayır (Kopya üzerinde)	Metoda salt okunur veri taşımak
ref	**Evet**	Hayır	**Evet (Kalıcı)**	Mevcut bir değişkeni içeride güncellemek
out	Hayır	**Evet (Zorunlu)**	**Evet (Fırlatır)**	Metottan birden çok sonuç üretip almak
in	Evet	**Yasak (Salt Okunur)**	Hayır	Büyük struct/veriyi kopyalamadan güvenle okumak

```
// Klasik Mülakat Örneği: İki sayının yerini değiştirme (Swap) void Swap(ref int a, ref int b) { int temp = a; a = b; b = temp; } int x = 5, y = 10; Swap(ref x, ref y); // Artık x = 10, y = 5 olur. ref olmasaydı x ve y değişmezdi!
```

10. Diziler Zaten Referans Tipidir: O Halde Neden `ref` Kullanırız?

En çok karıştırılan nokta burasıdır: Dizi referans tipi olduğu için elemanlarını değiştirmek `ref` gerektirmez; ancak **dizinin boyutunu değiştirmek veya yeniden new demek** kesinlikle `ref` gerektirir.

Hücre Değişimi (`ref` GEREKSİZDİR)

`void DiziGuncelle(int[] arr) { arr[0] = 99; }` çağrıldığında, Heap'teki mevcut diziye gidilip 0. eleman yazılır. Orijinal dizi anında değişir. `ref` kullanmak gereksizdir.

Boyutlandırma / `new` (`ref` ŞARTTIR)

Diziler sabit boyutludur. Boyut artırmak için `new T[N+1]` ile Heap'te yeni dizi açılır. Stack'teki asıl referansın bu yeni adresi göstermesi için `ref` zorunludur.

11. Manuel Dinamik Dizi Algoritması (Arka Plandaki `List ` Mantığı)

Aşağıdaki kod, C# dilinde hazır `Array.Resize` metodunun arka planda ne yaptığını ve `ref`'in neden hayati olduğunu gösterir:

```
void DiziyeElemanEkle(ref string[] kaynakDizi, string yeniEleman) { // 1. Adım: Heap üzerinde 1 eleman daha büyük YEPYENİ bir dizi alanı aç string[] yeniDizi = new string[kaynakDizi.Length + 1]; // 2. Adım: Eski dizinin verilerini yeni diziye güvenle kopyala Array.Copy(kaynakDizi, yeniDizi, kaynakDizi.Length); // 3. Adım: Yeni elemanı en son indekse yerleştir yeniDizi[yeniDizi.Length - 1] = yeniEleman; // 4. Adım (KRİTİK): ref sayesinde Main'deki değişken artık yeni diziyi gösterir! kaynakDizi = yeniDizi; } // Kullanım: string[] isimler = { "Ali", "Veli" }; DiziyeElemanEkle(ref isimler, "Ahmet"); Console.WriteLine(isimler.Length); // Çıktı: 3 (Eğer ref olmasaydı hala 2 kalırdı!)
```

12. Hazır `Array` Sınıfı Fonksiyonları ve `ref` İhtiyacı

Metot	Görevi	`ref` İster mi?	Neden?
`Array.Sort(arr)`	Diziyi alfabetik / sayısal sıralar	Hayır	Mevcut Heap hücrelerinin yeri değişir, yeni dizi açılmaz.
`Array.Reverse(arr)`	Diziyi ters çevirir	Hayır	Mevcut hücreler kendi içinde takas edilir.
`Array.IndexOf(arr, val)`	Elemanın ilk indeksini arar	Hayır	Sadece hücreleri okur, adrese dokunmaz.
`Array.Copy(s, d, len)`	Kaynak hücreleri hedefe taşır	Hayır	Verilen hedef dizinin hücrelerine yazar.
`Array.Resize(ref arr, n)`	Dizi boyutunu değiştirir	**EVET**	Heap'te yeni dizi açıp değişkeni o adrese bağlamak zorundadır.

Akılda Tutulacak Altın Kural: Eğer metot dizinin *içindeki kutuları* boyuyorsa `ref` gerekmez. Eğer metot diziyi *kökünden söküp yerine yeni bir dizi getiriyorsa* `ref` şarttır.